MS76 Qualification of existing runtime gate paths

Status: implemented for the explicit scope in `factory/verifier-qualification/runtime-gates.json`.

The earlier qualification kit tested a separate development comparator. MS76 adds an executable campaign that calls existing CloudBank business assertions and the existing CardDemo comparison CLI. The campaign fails if a declared correct case is rejected, a critical witnessed defect escapes or cannot be decided, a missing-evidence challenge passes, or the bound source changes during execution.

```
PYTHONPATH=src python3 -m lightyear_qualification runtime-gates \
  --output work/verifier-qualification/ms76
PYTHONPATH=src python3 -m unittest tests.test_runtime_gate_qualification -v
```

The output path must be new. `report.json` retains raw observations, gate results, fault witnesses, source identities and exclusions; `report.md` summarizes them. Exit 0 means this declared campaign passed, 1 means qualification failed, and 2 means the command could not complete valid evaluation. CI runs on Linux and Windows and retains the reports, including failed campaign reports.

CloudBank gate

The campaign invokes the unmodified `Journeys.success`, `invalid`, `insufficient`, `concurrent` and `account_restart` methods from `lightyear_data.cloudbank_journeys`, which the live journey runner also uses. Both SQLite/integer and journal/Decimal targets execute in real local processes. Each request starts a fresh process with persisted state. Concurrent requests are batched into the target's actual concurrent command and execute with workers inside that process; this does not test independent distributed writers.

Ten correct cases cover five methods on two implementations. Three executable faults exercise wrong recipient, lost operation history, and lost concurrent updates. Separate read-only SQLite queries witness persisted defects without calling the adapter, normalizer or gate. A nontriggering rounding fault is retained as not-exercised and excluded from the detection denominator. Missing observations are indeterminate and cannot pass.

The adapter projects target account observations and transfer operations into the journey interface. One target cent maps exactly to one integer journey unit. Operations project into paired withdrawal/deposit journal entries. This projection does not establish native participant-journal completeness or independently authenticate observations. HTTP statuses are translations of target outcomes; fixture markers are adapter context, not evidence of actual HTTP/auth boundaries.

Only five of the eighteen existing journey scenarios are exercised. Authentication, service readiness, customer access, messaging, deposits, clearance, credit/chat, dependency recovery and full-stack restart remain outside this campaign. Decimal rounding, dates and transfer replay from the development corpus are not asserted by these selected methods. The report does not label the whole CloudBank gate qualified and does not create MS65/MS66/MS67 live execution receipts.

CardDemo gate

The real local oracle produces baseline and candidate fixed-width outputs. The campaign passes them through `carddemo_oracle.cli.main(compare)`, preserving the actual comparator report and exit code. Correct identical outputs and permitted record ordering/filler alternatives must pass.

Seven persisted output mutations challenge balances, amounts, card identity, dates, business description, duplicated records and missing records. Separate copybook byte-range and record-count witnesses confirm the changes. These are file mutation experiments, not independently implemented

defective applications. Missing/malformed files and two empty lanes are indeterminate; a one-sided empty candidate is a detected mismatch. This does not execute native COBOL or Java.

Qualification evidence and boundaries

The tests disable the actual CloudBank journal assertion and CardDemo comparator in turn. The campaign then reports witnessed defects as missed and qualification fails. This demonstrates that the existing gates, rather than an additional development comparator, determine acceptance in this campaign.

All repository Python sources and the declared gate/normalization contracts are hashed before and after the campaign. Correct cases, detected, blocked, missed, unexercised faults and scope exclusions remain distinct. The CloudBank and CardDemo detection denominators describe different experiments and are not combined into a statistical estimate. Both implementations and challenges share an author.

No partner compatibility, independent blind validation, whole-application gate qualification, fresh native runtime execution or production readiness is claimed. Independent observation, broader interfaces and private reviewer challenges remain subsequent milestones. This is a Markdown implementation record.